

# Comunicação por Computador

## Trabalho Prático 2

Dezembro 2025



**Universidade do Minho**  
Escola de Engenharia

**Grupo: 10**

**Realizado por:** Sofia Margarida Rodrigues Freitas, a106798  
Soraia Filipa Ribeiro Pereira, a106806

# 1. Introdução

Este trabalho prático tem como objetivo conceber e implementar uma solução de comunicação para um cenário de exploração planetária, envolvendo uma Nave-Mãe, múltiplos rovers e uma interface de supervisão designada por Ground Control. A solução integra dois protocolos principais: TelemetryStream (TS), responsável pela monitorização contínua do estado dos rovers, e MissionLink (ML), dedicado à definição, envio e acompanhamento de missões.

Ao longo do relatório descrevem-se as decisões de arquitetura, o desenho e implementação dos protocolos de comunicação, os mecanismos de controlo de erros e fiabilidade, bem como a API de observação e a interface de Ground Control utilizada para monitorizar, em tempo quase real, o comportamento da frota e o progresso das missões.

Como linguagem de programação optámos por Python devido à familiaridade da equipa com a linguagem e à sua simplicidade, que facilitou a implementação dos conceitos abordados.

## 2. Topologia e Arquitetura da solução

### 2.1 Topologia

A topologia de rede desenvolvida para simulação encontra-se representada na figura 1. A Nave-Mãe atua como o nó central, ao qual se conectam todos os outros intervenientes, sejam os Rovers ou a interface de observação, Ground Control. De forma a testar os protocolos desenvolvidos em cenários de perda e latência, adicionaram-se os quatro satélites entre os rovers 3 e 4 e a nave-mãe e configuraram-se certas ligações da seguinte forma:

- rover-03 - n7: 15% de perdas e 80ms de latência;
- rover-04 - n10: 30% de perdas e 150ms de latência;
- n10 - n7 e n10 -n9: 10% de perdas.

Para garantir o correto encaminhamento do tráfego para diferentes sub-redes, desenvolveu-se o script *mother\_route\_setup* do módulo *config*. Para fácil inicialização de todos os componentes do sistema também foi desenvolvido o script *setup*, cujas instruções para execução encontram-se no ficheiro *README.md*.

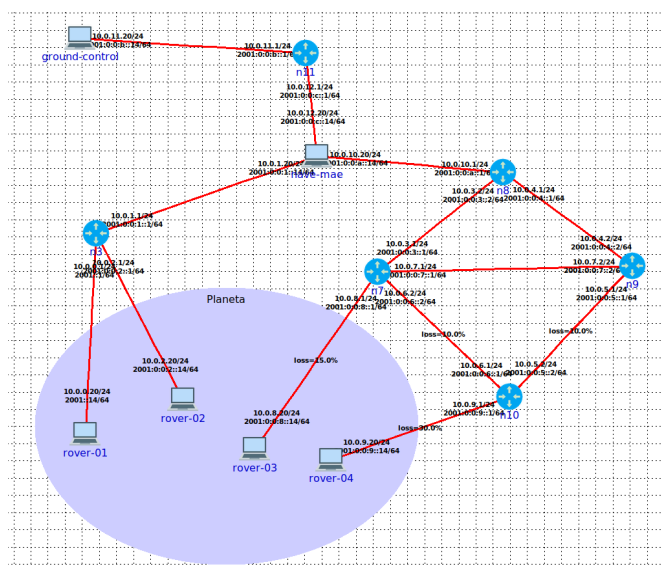


Figura 1: Topologia de Rede

## 2.2 Dados de Monitorização - TelemetryStream

Para garantir a observabilidade do estado dos rovers em tempo real, foi implementado o protocolo TelemetryStream (TS). Para facilitar a serialização destes dados, a telemetria foi desenhada para ser leve e conter apenas a informação essencial para a reconstrução do estado do Rover no lado do servidor.

A estrutura de dados da telemetria é composta pelos seguintes parâmetros fundamentais:

- Rover ID: Identificador único da unidade.
- State: O estado operacional atual, que pode assumir os valores IDLE (em repouso), ACTIVE (a executar tarefa), ON\_THE\_WAY (em deslocação para o objetivo) ou ERROR.
- Posição: Coordenadas polares atuais do rover.
- Velocidade: Velocidade instantânea de deslocação (km/h).
- Direção: Direção atual do movimento em graus.
- Nível de energia: Nível de bateria restante (0-100%).
- Orientação solar: Orientação dos painéis solares ao sol representado em coordenadas angulares (direção, elevação), utilizada internamente para calcular a eficiência de recarga de energia baseada na posição simulada do sol.
- Temperaturas: Monitorização da temperatura externa/ambiente e temperatura interna (sistema), cujas variações são simuladas com base na hora do dia e no esforço do motor (velocidade e atividade).

## 2.3 Definição de Missão - MissionLink

No entanto, o controlo operacional do Rover é gerido através do MissionLink (ML), um protocolo responsável pela definição e envio de missões. A implementação da estrutura de uma Missão inclui os seguintes campos obrigatórios para a sua validação e execução:

- Identificador de Missão: Um identificador único que permite rastrear a missão desde a criação no Ground Control até à conclusão no Rover.
- Area: Duas coordenadas polares do tipo  $[[r1, a1], [r2, a2]]$ , onde  $r$  é distância em km e  $a$  é ângulo em graus, que definem a região de operação da missão.
- Tarefa: O tipo de tarefa designada para aquela missão, que pode ser: Sample collection, Image Capture, Environmental Analysis, Equipment Deployment, Mapping ou Auto-Diagnostic. Todos estes são representados na missão pelas iniciais, sendo respetivamente: S,I,E,D,M,A.
- Parâmetro: Parâmetros adicionais para cada missão, que especificam a “task” com ordens mais diretas. Assim sendo, para cada tipo temos:
  - **Sample collection:** “S:Soil”, “R:Rock”, “A:Atmosphere”, “W:Water” (tipo de sample).
  - **Image Capture:** “P:Panoramic”, “C:Infrared”, “M:Macro” (tipo de imagem).
  - **Environmental Analysis:** “T”: 'Temperature', 'R': 'Radiation', 'P': 'Pressure', 'H': 'Humidity” (parâmetro ambiental a analisar).
  - **Equipment Deployment:** “S”: 'Seismometer', 'W': 'Weather Station', 'M': 'Marker', 'C': 'Communication Relay” (equipamento a ser instalado).
  - **Mapping:** “H”: 'High', 'M': 'Medium', 'L': 'Low” (nível de detalhe).
- Duração: duração total prevista da missão, em segundos.

- Intervalo entre atualizações: Intervalo entre atualizações de estado de missão, em segundos.

## 3. Protocolos de Comunicação entre o Rover e a Nave-Mãe

### 3.1. Telemetry Stream

O Telemetry Stream (TS) implementa um servidor TCP na nave-mãe (porta `TS_DEFAULT_PORT=5555`) que aceita conexões unidirecionais dos rovers. Cada rover estabelece uma ligação TCP persistente com este servidor e envia a cada 5 segundos os seus dados de telemetria, descritos anteriormente.

#### 3.2.2. Serialização e Desserialização de mensagens do Telemetry System

Cada mensagem do Telemetry Stream é serializada na origem e desserializada no destino, de modo a garantir que esta é transmitida de uma forma compacta. Para assegurar que as mensagens sejam reconstruídas corretamente após a sua receção, definiu-se uma estrutura fixa para estas, representada na tabela 1, que especifica para cada campo o tipo de dados usados para a sua representação e o respetivo tamanho em bytes após serialização.

Para serialização de *Strings*, utilizou-se o método `encode('utf-8')` da biblioteca standard do *Python*, que segue o padrão de codificação UTF-8. Já para a serialização dos *floats* recorreu-se aos métodos `pack()` e `unpack()` do módulo *struct* da biblioteca do *Python*, uma vez que, de acordo com a sua documentação oficial, este utiliza o padrão IEEE 754 para representação de floats - “The ‘f’ and ‘d’ conversion codes pack and unpack in IEEE 754 binary32 and binary64 formats, respectively.” (<sup>1</sup> Python Software Foundation. 2023. *struct*). É assim garantido que estas mensagens podem ser desserializadas em qualquer linguagem com suporte a estes padrões, sem serem utilizados bytes adicionais de codificação ou metadados.

De forma a reduzir o overhead de transmissão, o identificador do rover foi serializado utilizando apenas os dois dígitos numéricos (formato *R-XX*). Utilizou-se também uma string com três caracteres em vez de um int para a serialização do nível de bateria (‘000’ a ‘100’). Embora o estado operacional dos rovers seja originalmente representado por uma string de tamanho variável, optou-se ainda por criar um mapa que atribui a cada estado, definido anteriormente, um código representado por um só carácter - tabela 2.

| Campo               | Tipo    | Tamanho (bytes) |
|---------------------|---------|-----------------|
| Rover ID            | String  | 2               |
| Posição             | Float[] | 8               |
| Nível de Bateria    | String  | 3               |
| Orientação Solar    | Float[] | 8               |
| Temperatura Externa | Float   | 4               |

|                     |        |    |
|---------------------|--------|----|
| Temperatura Interna | Float  | 4  |
| Velocidade          | Float  | 4  |
| Direção             | Float  | 4  |
| Estado Operacional  | String | 1  |
| Total               |        | 38 |

**Tabela 1:** Estrutura das mensagens do Telemetry Stream

| Estado Operacional | Código |
|--------------------|--------|
| ACTIVE             | A      |
| ON_THE_WAY         | W      |
| IDLE               | I      |
| ERROR              | E      |

**Tabela 2:** Códigos para serialização do estado operacional

## 3.2. Mission Link

### 3.2.1. Design do protocolo

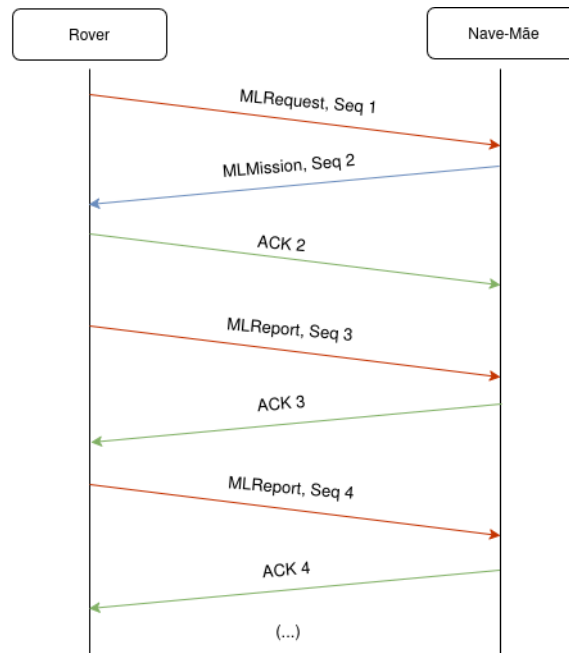
Para cada uma das mensagens do protocolo Mission Link (ML), desenvolveu-se um cabeçalho contendo o número de sequência da mensagem e o timestamp do seu envio. A informação compreendida no payload varia conforme o respetivo tipo de mensagem, que pode ser um dos seguintes:

- Pedidos (classe *MLRequest*): correspondem aos pedidos de missão dos rovers à nave-mãe. Indicam o identificador do rover emissor da mensagem.
- Missões (classe *MLMission*): são os pacotes enviados pela nave-mãe em resposta aos pedidos, que contêm os parâmetros da missão, incluindo o seu identificador, o nome da tarefa e, caso exista para essa tarefa, um parâmetro adicional, a área de atuação, a duração e o intervalo entre as atualizações.
- Atualizações (classe *MLReport*): atualizações do progresso da missão enviadas após o rover chegar à área da missão. Indicam o identificador da missão, o progresso da missão em percentagem e o último evento ocorrido.
- ACKs (classe *MLAck*): usados para a confirmação de receção de mensagens.

Os números de sequência são únicos dentro do contexto de cada missão, começando em 1 para o pedido e incrementando sequencialmente para cada nova mensagem relativa à missão atribuída (*MLMission* com *seq=2*, primeiro *MLReport* com *seq=3*, etc). Os ACKs mantêm o mesmo número da mensagem que estão a confirmar.

É utilizada a porta 4444 (*MS\_DEFAULT\_PORT*) na nave-mãe como endpoint UDP para receção de pedidos dos rovers. As missões enviadas pela nave-mãe são geradas dinamicamente através dos métodos da classe *MissionGenerator*: se, segundo a sua última telemetria, o rover estiver em estado de erro, é atribuída a tarefa de auto-diagnóstico. Caso

contrário, é escolhida aleatoriamente uma das tarefas definidas. Os restantes parâmetros da missão (distância, duração, intervalo de entre atualizações) são gerados aleatoriamente dentro dos limites especificados no ficheiro de configuração *mission\_parameters.json* no módulo *config*, que podem ser alterados.



**Figura 2:** Diagrama de sequência da comunicação do ML num cenário sem perdas

### 3.2.2. Mecanismos de Controlo

De modo a garantir o controlo de erros e a fiabilidade na comunicação sobre UDP, foram implementados neste protocolo os mecanismos de controlo descritos abaixo.

- **Confirmação de pacotes:** Cada pacote enviado deve ser confirmado pelo destinatário. Esta confirmação é feita através do envio de mensagens do tipo ACK para o caso das mensagens do tipo *MLMission* e *MLReport*. Relativamente aos pedidos de missão, é enviada como forma de confirmação a mensagem de especificação da missão - *MLMission*.
- **Retransmissões:** Caso um pacote não seja confirmado após um definido intervalo de tempo (*timeout*), ele é retransmitido. Os pacotes podem ser retransmitidos só até a um limite máximo de tentativas permitido. No caso dos pedidos, o rover fica bloqueado até receber uma missão da nave-mãe. Se não receber nenhuma confirmação depois de atingir o limite de retransmissões, entra num estado de *backoff*, esperando 10 segundos antes de enviar um novo pedido. Já para os outros casos, é mantido um mapa com as mensagens pendentes de confirmação, em que a chave é um tuplo com o id da missão e o número de sequência e o valor contém o respetivo pacote, o número de retransmissões feitas e o endereço de destino. Existe uma thread responsável pela retransmissão e atualização do estado destas mensagens.
- **Controlo de Pacotes Duplicados:** Cada pacote recebido é verificado através do identificador de missão e número de sequência. Se um pacote duplicado for detetado, o sistema descarta-o mas retransmite o ACK correspondente, assegurando que mesmo

quando um ACK se perde na rede, o emissor eventualmente recebe confirmação através da retransmissão provocada pela receção do duplicado.

- **ACKs cumulativos:** Cada ACK confirma a receção de todos os pacotes relativos à respetiva missão até ao seu número de sequência (ex: ACK com número de sequência 5 confirma todos os pacotes dessa missão pendentes de confirmação com número de sequência até 5).

### 3.2.3. Serialização e Desserialização das mensagens do Mission Link

Tal como procedido para os pacotes do Telemetry System, foi feita a serialização na origem e desserialização no respetivo destino para os pacotes do Mission Link através de métodos como *encode('utf-8')* para Strings e *struct.pack()* para floats. Para inteiros utilizou-se *to\_bytes()/from\_bytes()* (32-bit, *big-endian*). Nas tabelas 1 e 2 encontram-se representados os campos incluídos no cabeçalho e no payload de cada mensagem, o seu tipo de dados e o seu tamanho em bytes.

De modo a garantir uma melhor compactação dos dados dos pacotes, tal como feito para os identificadores dos rovers, foi feita a serialização apenas do valor numérico dos identificadores das missões (*M-XXX*). Adicionalmente, para a implementação dos tipos de missões descritas na secção 2.3, foi criado um dicionário presente em *common/mission\_types.py* com códigos de um carácter para identificação das tarefas, dos seus parâmetros e eventos de progresso. Para o *MLReport* recorreu-se a esse código para a serialização do último evento ocorrido. Já no caso das mensagens *MLMission*, para compactar ainda mais o tamanho dos pacotes, foi usado um byte para especificar os códigos da tarefa e do seu parâmetro, onde os bits 7 a 5 identificam o código da missão (6 missões no total), o bit 4 indica a presença ou ausência de um parâmetro adicional, e os bits 3 a 2 o respetivo parâmetro, caso este exista (uma missão contém até 4 parâmetros).

| Campo               | Tipo                | Tamanho (bytes) |
|---------------------|---------------------|-----------------|
| Número de sequência | Int                 | 4               |
| Timestamp           | Int                 | 4               |
| Tipo de Mensagem    | String (1 carácter) | 1               |

**Tabela 3:** Estrutura do cabeçalho de uma mensagem Mission Link

| Tipo de Mensagem | Campo                        | Tipo    | Tamanho (bytes) |
|------------------|------------------------------|---------|-----------------|
| MLRequest        | Rover ID                     | String  | 2               |
| MLMission        | ID de missão                 | String  | 3               |
|                  | Área                         | Float[] | 16              |
|                  | Tarefa e Parâmetro           | Byte    | 1               |
|                  | Duração                      | Int     | 4               |
|                  | Intervalo entre atualizações | Int     | 4               |
| MLReport         | ID de missão                 | String  | 3               |

|       |                  |        |     |
|-------|------------------|--------|-----|
|       | Evento de missão | String | 1   |
|       | Progresso        | String | 1-3 |
| MLAck | ID de missão     | String | 3   |

**Tabela 4:** Estrutura dos payloads das mensagens do Mission Link

## 4. Interface de observação de Ground Control

A interface de Ground Control funciona como centro de monitorização da frota de rovers gerida pela Nave-Mãe, permitindo ao utilizador acompanhar o estado dos rovers, a telemetria disponível e o progresso das missões em execução. A comunicação é feita através de uma API de observação exposta pela Nave-Mãe, que disponibiliza informação consolidada em formato JSON, abstraindo a UI dos detalhes de armazenamento interno, que são geridos de forma transparente pela Nave-Mãe.

### 4.1. API de observação da Nave-Mãe

A Nave-Mãe expõe uma API REST sobre HTTP através de um servidor HTTP baseado em `HTTPServer` e num request handler (*MotherRequestHandler*), que fornece um conjunto de endpoints de leitura para Ground Control. Estes endpoints encapsulam o acesso à *Database* da Nave-mãe, a qual mantém em memória o estado dos rovers, a telemetria mais recente recebida e a lista de missões registadas, atualizadas a partir das mensagens ML (*MLMission*, *MLReport*).

Os principais endpoints de observação são acedidos através do método HTTP GET e incluem suporte CORS (para permitir acesso via browser). No nosso projeto, optámos pelos seguintes endpoints:

- `/rovers/active`

Devolve a lista de rovers conhecidos e o respetivo estado atual, conforme mantido na estrutura `dados["rovers"]` da *Database*.

- `/missions`

Devolve um dicionário com todas as missões conhecidas, indexadas por `mission_id`, com os campos mencionados anteriormente no relatório. Estes dados são atualizados na base de dados pelos métodos *add\_mission* (quando chega uma mensagem *MLMission*) e *update\_mission* (quando chega um *MLReport*).

- `/telemetry`

Devolve a última telemetria conhecida de cada rover, a partir da estrutura `'last_telemetries'` gerida por *Database.register\_telemetry*. Se ainda não existir qualquer telemetria registada, o servidor devolve um erro JSON com mensagem indicando a ausência de dados e código HTTP 404.

## 4.2. Ground Control

A camada de Ground Control foi implementada como uma aplicação web estática, composta por várias páginas HTML e JavaScript (por exemplo, `missions.html`, `telemetry.html`, `rovers.html`). Estas páginas consomem diretamente a API HTTP da Nave-Mãe através de chamadas `fetch()` ao backend (por exemplo, `/missions`, `/telemetry` e `/rovers/active`), recebendo dados em formato JSON. No próprio JavaScript do browser, estes dados são convertidos em objetos e usados para construir e atualizar os elementos visuais de monitorização, como listas de rovers ativos, tabelas de missões com filtros por estado, e dados de telemetria atualizados a cada 5 segundos. Para aceder à interface web do Ground Control após a execução do script de inicialização referido em 2.1, basta executar `firefox ground/index.html` num `bash` do host `ground-control`.

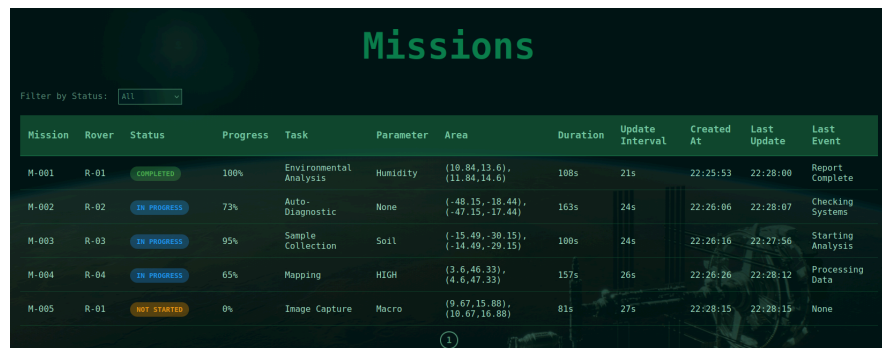


| Rover | Position         | State  | Speed  | Direction | Power | Solar Orientation | External Temperature | Internal Temperature |
|-------|------------------|--------|--------|-----------|-------|-------------------|----------------------|----------------------|
| R-01  | (11.34, 14.1)    | ACTIVE | 0 km/h | -113.62°  | 97%   | (336.47°, 0°)     | -7.33°C              | 7.67°C               |
| R-02  | (-47.65, -17.94) | ACTIVE | 0 km/h | -27.61°   | 97%   | (336.53°, 0°)     | -7.19°C              | 7.81°C               |
| R-03  | (-14.99, -29.65) | ACTIVE | 0 km/h | -125.4°   | 98%   | (336.58°, 0°)     | -7.85°C              | 7.15°C               |
| R-04  | (4.1, 46.83)     | ACTIVE | 0 km/h | 71.78°    | 99%   | (336.61°, 0°)     | -7.87°C              | 7.13°C               |

Last update: 22:27:01

< back

Figura 3: Página da API que demonstra a última telemetria dos rovers ativos



| Mission | Rover | Status      | Progress | Task                   | Parameter | Area                             | Duration | Update Interval | Created At | Last Update | Last Event        |
|---------|-------|-------------|----------|------------------------|-----------|----------------------------------|----------|-----------------|------------|-------------|-------------------|
| M-001   | R-01  | COMPLETED   | 100%     | Environmental Analysis | Humidity  | (10.84,13.6), (11.84,14.6)       | 108s     | 21s             | 22:25:53   | 22:28:00    | Report Complete   |
| M-002   | R-02  | IN PROGRESS | 73%      | Auto-Diagnostic        | None      | (-48.15,-18.44), (-47.15,-17.44) | 163s     | 24s             | 22:26:06   | 22:28:07    | Checking Systems  |
| M-003   | R-03  | IN PROGRESS | 95%      | Sample Collection      | Soil      | (-15.49,-30.15), (-14.49,-29.15) | 108s     | 24s             | 22:26:16   | 22:27:56    | Starting Analysis |
| M-004   | R-04  | IN PROGRESS | 65%      | Mapping                | HIGH      | (3.6,46.33), (4.6,47.33)         | 157s     | 26s             | 22:26:26   | 22:28:12    | Processing Data   |
| M-005   | R-01  | NOT STARTED | 0%       | Image Capture          | Macro     | (9.67,15.88), (10.67,16.88)      | 81s      | 27s             | 22:28:15   | 22:28:15    | None              |

1

Figura 4: Página da API que demonstra as missões, e os seus respectivos campos

## 5. Testes Realizados e Resultados

Como testado a partir da API de observação da nave-mãe e observável nas figuras 3 e 4, tanto a telemetria dos rovers quanto o estado das missões ativas são atualizados com sucesso ao longo da execução do sistema, o que demonstra a capacidade da nave-mãe de manter e processar múltiplas conexões Telemetry Stream e pedidos do Mission Link de forma concorrente.

Para testar mais concretamente a devida execução do protocolo desenvolvido para o Mission Link, foi executada uma missão no rover-01, cuja ligação à nave-mãe não apresenta perdas nem latência adicional. Na tabela 5 encontram-se partes dos logs obtidos para a execução desta

missão. Verifica-se que a comunicação ocorreu de forma consistente com o protocolo, com todos os pacotes e respetivos ACKs a serem trocados com sucesso.

| mothership_log.txt                                                                 | R-01_log.txt                                               |
|------------------------------------------------------------------------------------|------------------------------------------------------------|
| [21:42:46.745] Received MLRequest, seq=1: From R-01 (10.0.0.20:56452)              | [21:42:46.745] Sent MLRequest, seq=1 to 10.0.1.20:4444     |
| [21:42:46.745] Sent MLMission, seq=2: M-001, To: R-01                              | [21:42:46.745] Received MLMission, seq=2                   |
| [21:42:46.745] Received MLAck for Mission ID: M-001, ack=2                         | [21:42:46.745] Sent MLAck for Mission: M-001, ack=2        |
| [21:42:47.747] Received MLReport for Mission:M-001, seq=3: Progress: 0, Status: 1  | [21:42:47.747] Sent MLReport for Mission:M-001, seq=3      |
| [21:42:47.747] Sent MLAck for Mission: M-001, ack=3                                | [21:42:47.748] Received MLAck for Mission ID: M-001, ack=3 |
| [21:42:57.758] Received MLReport for Mission:M-001, seq=4: Progress: 18, Status: 1 | [21:42:57.757] Sent MLReport for Mission:M-001, seq=4      |
| [21:42:57.758] Sent MLAck for Mission: M-001, ack=4                                | [21:42:57.758] Received MLAck for Mission ID: M-001, ack=4 |

**Tabela 5:** Logs da missão M-001 (Rover-01, 0% perda)

A mesma missão foi repetida para o rover-04, cuja ligação à nave-mãe apresenta 30% de perdas e 150ms de latência no primeiro salto e 10% de perdas no segundo, como descrito em 2, de forma a avaliar a robustez do protocolo em casos de perdas e latência. Os logs deste teste (tabela 6) demonstram de forma clara os mecanismos de controlo implementados, sendo possível observar a retransmissão dos pacotes depois de não terem sido recebidos os pacotes de confirmação (MLMission e MLAck) após um intervalo de tempo igual ao timeout.

| mothership_log.txt                                                                 | R-04_log.txt                                                            |
|------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| [22:07:57.018] Received MLRequest, seq=1: From R-04 (10.0.9.20:46381)              | [22:07:51.153] Sent MLRequest, seq=1 to 10.0.10.20:4444                 |
| [22:07:57.019] Sent MLMission, seq=2: M-001, To: R-04                              | [22:07:54.156] Retransmitting MLRequest (1/3)                           |
| [22:08:00.020] Retransmitting packet from mission M-001, seq=2, retry 1            | [22:07:57.160] Retransmitting MLRequest (2/3)                           |
| [22:08:01.122] Received MLAck for Mission ID: M-001, ack=2                         | [22:08:00.151] Received MLMission, seq=2                                |
| [22:08:08.324] Received MLReport for Mission:M-001, seq=3: Progress: 0, Status: 1  | [22:08:00.972] Sent MLAck for Mission: M-001, ack=2                     |
| [22:08:08.325] Sent MLAck for Mission: M-001, ack=3                                | [22:08:01.975] Sent MLReport for Mission:M-001, seq=3                   |
| [22:08:15.332] Received MLReport for Mission:M-001, seq=4: Progress: 18, Status: 1 | [22:08:05.169] Retransmitting packet from mission M-001, seq=3, retry 1 |
| [22:08:15.333] Sent MLAck for Mission: M-001, ack=4                                | [22:08:08.173] Retransmitting packet from mission M-001, seq=3, retry 2 |
|                                                                                    | [22:08:08.476] Received MLAck for Mission ID: M-001, ack=3              |
|                                                                                    | [22:08:11.981] Sent MLReport for Mission:M-001, seq=4                   |
|                                                                                    | [22:08:15.182] Retransmitting packet from mission M-001, seq=4, retry 1 |
|                                                                                    | [22:08:15.483] Received MLAck for Mission ID: M-001, ack=4              |

**Tabela 6:** Logs da missão M-001 (Rover-04, 37% perda, 150ms latência)

Durante o desenvolvimento deste protocolo, foram ainda realizados testes para que fosse possível obter os valores ótimos para o timeout e o número máximo de retransmissões referidos na secção 3.2.2. Para tal, consideramos missões com uma duração de 100 segundos e um intervalo de 10 segundos entre atualizações, cuja área incluísse a posição atual de rover, para não considerar o tempo perdido em deslocação. Esta missão foi repetida 3 vezes no o rover 01 para controlo e nos rovers 03 e 04 de modo a considerar diferentes níveis de perda e latência. Os valores médios dos resultados obtidos nas várias execuções encontram-se na tabela 5.

| (Timeout, N° max Retransmissões) | Rover | Retransmissões | Tempo (s) | Timed out | Backoffs |
|----------------------------------|-------|----------------|-----------|-----------|----------|
| -                                | R-01  | 0              | 111.110   | 0         | 0        |
| (1s, 4)                          | R-03  | 7              | 114.197   | 0         | 0        |
|                                  | R-04  | 15             | 119.786   | 2         | 1        |
| (3s,3)                           | R-03  | 5              | 114.358   | 0         | 0        |
|                                  | R-04  | 12             | 119.567   | 1         | 0        |
| (5s,3)                           | R-03  | 4              | 117.214   | 0         | 0        |
|                                  | R-04  | 10             | 121.559   | 0         | 0        |
| (3s,1)                           | R-03  | 3              | 111.350   | 2         | 0        |
|                                  | R-04  | 7              | 116.476   | 4         | 1        |
| (3s, 5)                          | R-03  | 5              | 118.357   | 0         | 0        |
|                                  | R-04  | 22             | 121.559   | 1         | 0        |

**Tabela 5:** Resultados obtidos para diferentes valores de timeout e número máximo de retransmissões

Analisando os valores da tabela, concluímos que para valores de timeout baixos (1.0 segundo) e para números máximos de retransmissões reduzidos, embora sejam obtidos tempos de execução relativamente mais baixos, há mais ocorrências de pacotes descartados (timed out), ou seja, que não foram confirmados após o número máximo de retransmissões, assim como backoffs para o envio de pedidos. Já para 5 segundos de timeout, embora não haja pacotes descartados, verifica-se um tempo maior perdido em processamento para retransmissões. O mesmo acontece para um número máximo de retransmissões igual a 5. Assim, concluiu-se que 3.0 segundos e 4 seriam, respetivamente, os valores mais ajustados para o timeout e o número máximo de retransmissões para o cenário de teste desenvolvido.

## 6. Conclusões

Durante a realização deste trabalho encontraram-se algumas limitações principalmente no que diz respeito ao desenvolvimento do Mission Link. A ausência de implementação de outros mecanismos de controlo mais avançados como o ajuste da janela em resposta a diferentes níveis de congestão de rede resultam num maior número de erros para cenários com um maior número de rovers e menores intervalos de tempo entre envio de pacotes. Sendo assim, como melhoria futura poderia-se implementar mecanismos de controlo de congestão semelhantes ao *slow start* e *congestion avoidance* do TCP para o Mission Link. Outra seria a adaptação do loop de receção de dados para suportar pacotes de tamanho variável, uma vez que, embora não seja necessário atualmente devido ao tamanho reduzido e fixo das mensagens definidas para o ML, ofereceria mais liberdade e facilidade para a extensão do protocolo em situações futuras.

No que diz respeito à implementação da telemetria dos rovers e das missões, tivemos dificuldades em encontrar um meio termo entre realismo e usabilidade para testes. Por exemplo, a implementação de dimensões mais realistas para o mapa (50kmx50km) levou a que fosse necessário o uso de uma

velocidade máxima relativamente irrealista para os rovers (80km/h) para que fosse possível testar num tempo de simulação razoável.

## Referências

<sup>1</sup> Struct - interpret bytes as packed binary data. Python documentation. (n.d.). Retirado a 5 de dezembro de 2025, de <https://docs.python.org/3/library/struct.html>